

was not obstructed (Figure 4(b)).

We trained a V-DCG grounding model following a procedure similar to (?): collect NLI using crowdsourcing, annotate a training corpus, and design features for training a model using the corpus. To collect the training corpus, we executed each specification in eight different simulated environments with varying numbers of blocks and bin behaviors (16 environments in total for the two tasks). For each environment we used manually defined groundings for the propositions, synthesized a controller, and rendered videos of a simulated Baxter executing the task.

We then crowdsourced NLI through surveys¹ that showed videos of the robot’s execution and required users to freely type instructions that would produce that behavior. The responses consisted of multi-sentence instructions e.g. “Pick up the innermost blue cube and drop into the right box. Then pick up the remaining blue cube and the innermost red cube. Drop the red cube into the box on the left and then pick up the other red cube. Drop it into the left box and drop the blue one into the right.”

From the collected instructions we selected two corpora of multi-sentence instructions: 39 instructions for the sorting task and 21 instructions for the stacking task. We separated answers into sentences, parsed them using a production grammar and annotated the parsed instructions using the symbol hierarchy shown in Figure 2. The following are individual training instructions from the annotated training corpus:

Sorting:

“Pick up the remaining red cube and drop it in the left bin;”

“Pick up the blue block and put it in the box on your right;”

“Put the blue cube in the right bin and put the red cube in the left bin.”

Stacking:

“Take the red cube. Put the red cube on the green cube;”

“Take a green cube. Wait to remove blue cube. Put green cube on red cube.”

“Take the blue cube, lift the cube, then place it above the green cube.”

The two-task corpus of 60 instructions (21 stacking, 39 for sorting) contained 828 words (158 unique words) and expressed the following numbers of individual symbols: 153 ObjectType, 109 ObjectColor, 588 Object, 51 SpatialRelation, 60 Region, 275 Sensor, 223 Action, and 222 Scope. Our model covered (i.e. features could discriminate and represent annotations) 99.9872% of all the 1,132,710 factors in the joint corpus.

We evaluated V-DCG on the two-task corpus. Training and testing on all 60 instructions, the model could recover 42 instructions (17 for the stacking task and 25 for the sorting task). Training and testing separately, the model fully inferred 19/21 stacking instructions and 28/39 sorting instructions. Given that the current features offer good coverage of the factor space, as we expand these corpora additional data will improve performance on inferring full instructions. The

annotations between the two tasks have significant lexical, grammatical, and symbolic grounding differences; for example, “green” is only used in the stacking task, and “bin” is only used in the sorting task. However, the two tasks contain similar phrases referring to red or blue blocks. The current performance on the joint corpus is slightly lower than individually training models for each task due to an increase in size of the symbol space.

We deployed the trained model on a Rethink Robotics Research Baxter robot and implemented the interaction behavior shown in Figure 1 using open-source speech recognition² and text-to-speech³ software. The robot uses table-top segmentation of RGBD point-clouds to determine properties of the cubes on the table. The location of bins are pre-determined. A sampling-based motion planner was used to determine arm trajectories. Grasping of cubes on the final approach is performed by a controller using visual-servoing. Sample instructions are available in the supplemental video⁴.

CONCLUSION

In this paper we introduce a natural language interaction method which enables the robot to request the user to approve or reject automatically-generated environment assumptions. The interaction is triggered by synthesis outcomes which identify environment behaviors that would prevent the robot from accomplishing the task. LTL assumption formulae that restrict the environment from exhibiting these behaviors are generated automatically and then expressed through natural language in prompts which the user can accept or reject.

Grounding complex NLI to the physical environment and LTL formulae enables the synthesis of guaranteed controllers that can accomplish the task described in the instruction. Synthesis guarantees robot behavior under assumptions describing constraints of the robot and the environment. If a grounding model is used in new environments, our contribution can identify new assumptions that will be needed.

Assumenda totam mollitia culpa delectus praesentium veniam aut id optio repudiandae, ex aliquid sint consequuntur quaerat molestiae magni quasi?Laudantium voluptatibus aut corporis, molestiae nemo atque quod fugiat velit, repellendus inventore odio veniam voluptatum magni eos, vel recusandae ducimus impedit repellendus in vero sed?Ad iusto quibusdam omnis eum non dicta, molestias error labore minima ab, consequatur nostrum commodi numquam eaque provident. Consequatur saepe atque dolores, voluptates non deleniti earum veniam temporibus, necessitatibus eaque fugit quod dolore, unde accusamus corrupti aliquam in debitis vero eveniet accusantium quisquam perspiciatis placeat?Repellat dicta voluptatum magni aspernatur inventore animi delectus quae ipsa, ratione quae impedit eaque, atque laborum recusandae quia reprehenderit ipsa eum deserunt adipisci repellendus preferendis, nulla officia provident architecto ex praesentium sapiente saepe possimus

¹We deployed identical surveys on <http://www.mturk.com> for the sorting task, and on <http://www.crowdflower.com> for the stacking task.

²<http://cmusphinx.sourceforge.net>

³<http://espeak.sourceforge.net/>

⁴<https://youtu.be/sG8dNGiYNNM>

error voluptatum.Pariatur commodi maxime sit inventore
placeat sequi fuga, illo non suscipit accusamus deleniti prae-
sentium fugit, totam sed vitae.